MANTIS Complete — A Unified Banking Multi-Agent Security, Compliance, and Observability Testbed

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—MANTIS (Modular Agent Network Testbed for Instrumentation and Security) is a configuration-driven security, compliance, and observability testbed built directly on a real banking multi-agent system. This paper consolidates the system's two prior papers – the original testbed [1] and its first set of post-paper extensions [2] – into a single, complete account of MANTIS as it exists today, and adds one further extension neither prior paper covers: a toggleable Anti-Money Laundering (AML) office, ported from the legacy Citi_P3 codebase MANTIS itself was originally extracted from [3]. The base system instruments an otherwise-unmodified front/mid/back-office banking agent system at five fixed control points (input, agent, interaction, tool, output) through a hook bus; an attack, failure, or defense plugin declares which point it targets entirely in YAML and is dispatched through the identical interface regardless of kind. Seven automated evaluator dimensions, a benchmarking suite that isolates instrumentation overhead from LLM sampling latency (median 0.19% of run duration), and a reproducibility layer recording a configuration hash, seed, and full environment per run round out the base testbed. On top of this, MANTIS now ships: six security-mechanism plugins covering every category a defense-in-depth testbed should (amount guardrail, risk-aware routing guard with an optional compliant-route recovery mode, batch-integrity guard, response redaction, rate limiting, action isolation); four additional observability exporters (Jaeger, Grafana, Phoenix, Langfuse); three additional banking workloads (dispute filing, SAR/exception escalation, loan pre-approval) plus a second institution profile; a schema-driven minimal browser UI whose every mutating action shells out to the same CLI a terminal user would invoke; an extended scenario library of 19 measured baselines and 26 attack/fault variants; and a real, toggleable integration of the independently-developed Zero Trust Backplane project's signed Policy Decision Point. This paper's own new contribution is the AML office: a fourth, off-by-default banking domain carrying real structuring detection (sliding-window cumulative-deposit analysis), mule-ring detection (closed-loop graph traversal over a customer's transaction history), identity-document verification (Levenshtein name scoring), and an alert-lifecycle state machine (open → investigation → cleared/escalated-to-SAR), all ported with the detection logic itself unchanged from Citi_P3, gated behind a dedicated aml_office: toggle that leaves every existing configuration's root agent byte-identical when absent. Live verification against a real model across five new scenarios surfaced, and let us fix, two real integration defects before reporting the extension as working: a routing-instruction gap that silently misrouted an identity-verification request to the wrong banking domain entirely, and an alert-lifecycle capability that was fully implemented and unit-tested yet unreachable by any agent in the built tree. After both fixes, all five AML scenarios execute correctly against the live backend, including a persisted, re-read-and-confirmed database state change from a live agent's own tool call. The full offline test suite stands at 853 passing tests with neither optional dependency installed, 880 with both the Zero Trust and AML optional dependencies installed, zero failures in every combination.

Index Terms—multi-agent systems, adversarial security, large language models, banking, anti-money laundering, zero trust, observability, benchmarking

Identify applicable funding agency here. If none, delete this.

I. Introduction

Large language model (LLM) agents are moving from single-turn assistants into orchestrated multi-agent systems that carry out multi-step business processes, and banking is a natural early target: recent work proposes multi-agent frameworks for financial anomaly detection [5] and credit assessment [6], each decomposing a task across specialized agents the way a bank already organizes front-, mid-, and back-office work. That same decomposition is also where new security risk enters – inter-agent communication is an exploitable surface in its own right [7], [8], and a broad survey of agentic security concludes most defenses evaluated in isolation can be bypassed by an adaptive attacker [9], [10]. No widely available testbed previously let a researcher take a realistic, domain-specific multi-agent system and run controlled adversarial experiments

against it without either reimplementing the orchestration framework or editing the business logic under test [11].

This paper is a consolidation, not a third independent contribution. The original MANTIS paper [1] built the hook bus, the banking runtime adapter, five attack and failure configurations, a seven-dimension evaluator, and a benchmarking suite – and named five extensions deliberately deferred to future work. A second paper [2] implemented all five: additional observability exporters, security-mechanism (defense) plugins, additional banking workloads and a second institution profile, a minimal schema-driven UI, and – after a team decision reversed an earlier exclusion – a real, toggleable integration of the independently-developed Zero Trust Backplane project [4]. Both papers' own artifacts (code, tests, and recorded evidence) remain frozen and unmodified; this paper neither edits nor supersedes them, it reports the complete system built on top of them, plus one further extension neither of them covers.

That further extension is an AML (Anti-Money Laundering) office. MANTIS's banking agents were originally extracted from a larger legacy monorepo, Citi_P3 [3], during the first paper's own WP1 migration; that legacy codebase had a working AML investigation capability – structuring detection, mule-ring detection, identity verification, SAR drafting, and alert triage – that never made the transition into MANTIS's modular architecture. This paper ports it, deliberately following the same discipline the Zero Trust integration established: a real capability adapted into the existing architecture, not reimplemented from a specification, gated behind a toggle so every pre-existing configuration is unaffected, and verified against the live system's own emitted trace rather than accepted on a clean exit code. Section VI reports that verification process finding two real integration defects before the extension could honestly be called complete, and the fixes for both.

The remainder of this paper is organized as follows. Section II summarizes the motivation and related work established in [1]. Section III describes the complete current architecture. Section IV summarizes the attack and failure plugin library and its live-verified evidence. Section V summarizes the six security-mechanism plugins and the Zero Trust Backplane integration. Section VI reports the AML office extension in full, the paper's own new contribution. Section VII summarizes additional banking workloads, institution profiles, the minimal UI, and the extended scenario library. Section VIII consolidates test and live-verification evidence for the complete system. Section IX concludes.

II. Background and Motivation

Multi-agent decomposition is a natural fit for banking because the workloads are already organized around specialized roles – monitoring, investigation, compliance sign-off, decision – and prior work demonstrates this pattern being actively explored in finance [5], [6]. Three properties of multi-agent LLM systems create risk a single-agent deployment does not have: agents ingest content produced by other agents, not only end users, so a compromised or spoofed agent's output can influence every downstream consumer [7]; routing and hand-off decisions are themselves attack surfaces, since an adversary who can intercept or relabel an inter-agent message can redirect a request to the wrong specialist or skip a required review step [8]; and tool invocation – the mechanism by which an agent actually moves money or posts a ledger entry – can be mutated in flight, a materially different threat than a model simply answering incorrectly. General red-teaming benchmarks for LLM agents [11] provide reusable attack scaffolding but evaluate against synthetic or generic tasks rather than an existing, production-shaped business workflow, and are not designed around the constraint that the business logic under test must remain completely unmodified – a constraint that matters because a testbed requiring edits to the fraud-review agent's own logic to add an experiment cannot be trusted to measure that agent's real behavior. Zero Trust architecture – never trusting a request implicitly, mediating every action through explicit policy [12], increasingly proposed specifically for agentic AI systems [13] – is the complementary governance layer Section V reports as a real, toggleable integration rather than the deliberately-deferred direction the first paper treated it as.

III. Architecture

Figure 1 shows the complete MANTIS pipeline as it stands today. An experiment is declared entirely in a YAML configuration; an orchestrator establishes a run identifier, seed, and lifecycle, then drives the banking workflow through a hook bus exposing five fixed control points – input, agent, interaction, tool, and output. Each dispatch carries a typed context (run id, trace id, workflow id, stage, source, target, payload, metadata) to every registered plugin whose declared stages match; a plugin returns continue, mutate, skip, deny, delay, or error, and a mutation threads correctly to the next plugin in the same dispatch while the fact that one occurred is recorded in context metadata, so a plugin dispatched later in the same chain – the observability plugin, by design, so it can observe every attack's effect – can distinguish a genuinely adversarial event from an ordinary one. Beneath the hook bus, a banking runtime adapter presents one stable interface (inventory(), workflows(), build()) over four domains: front office (transaction monitoring, fraud detection, compliance, customer service, dispute filing), mid office (operations planning, representative assist, loan pre-approval), back office (end-of-day reconciliation, settlement, SAR/exception escalation), and – new in this paper – AML office (structuring/mule-ring/identity-document investigation and SAR drafting, Section VI). The live inventory (mantis –inventory), introspected from

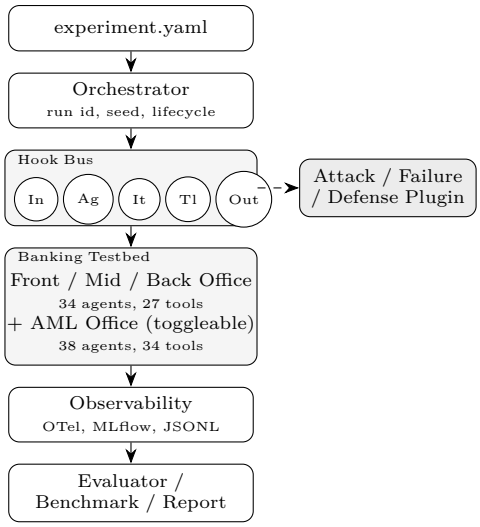


Fig. 1. The complete MANTIS pipeline. Every run passes through the same five-point hook bus regardless of banking domain; an attack, failure, or defense plugin attaches to the bus alongside the standard observability plugin. The AML office is a fourth banking domain, toggled per run, that does not change the pipeline shape when disabled.

the tool modules and agent registry rather than maintained as a hand-authored list, reports all four domains unconditionally – 38 agents and 34 tools (33 banking-domain tools plus the underlying agent framework's own routing tool) – since domain metadata registration is independent of any single run's configuration. The AML office's own toggle (Section VI) is a separate, narrower axis: it governs only whether a given run's actual agent tree instantiates the AML office's 4 agents and 7 tools, not whether the inventory can discover or validate against them. Every run built without the toggle – the only kind possible before this paper – uses exactly the 34-agent, 27-tool front/mid/back-office tree the two prior papers describe.

Every hook dispatch is mapped to one of seven typed event families (experiment, workflow, agent, interaction, tool, security, evaluation), exported as portable JSONL, OpenTelemetry spans, and an MLflow run sharing the same configuration hash and seed, with four additional OTLP-compatible exporters (Jaeger, Grafana Tempo, Phoenix, Langfuse) added as one-line configuration changes that fail silently rather than aborting a run when no collector is reachable. An automated evaluator reads a run's trace and reports trace completeness, tool-use correctness, workflow outcome, hook-point coverage, banking-domain coverage, artifact integrity, and attack ground truth – the last distinguishing a plugin that merely fired from one whose effect is corroborated against the configuration's own expected-tools/terminal-state baseline. Every run manifest records a SHA-256 configuration hash, the seed, the Python interpreter and dependency versions, the source commit, and – newly tracked – whether the run used a mock or live model and which model served it, so a reported result always traces back to the exact code, environment, and model that produced it.

IV. Attack and Failure Plugin Library

Five attack and failure configurations span all three always-on domains, registered through the identical plugin interface: prompt injection and message spoofing both intercept the interaction control point (an agent's outgoing call to the underlying model); route confusion intercepts a routing tool call and forces execution to a different downstream agent than the one the orchestration logic selected; tool-parameter mutation intercepts a tool call directly and rewrites its arguments, shipped against both a front-office funds transfer and a back-office ledger post; and a reliability-failure family (delay, timeout, malformed result) emits an ANOMALY security event rather than ATTACK_INJECTED, so an evaluator separates a security-caused anomaly from a routine fault by event type rather than inference. Live re-verification across five repeated trials each found route confusion unambiguous and total (5/5 fired, 5/5 ground-truth effect: the router's real transfer diverted to the chatbot workflow every trial, compliance never reached); tool mutation (front) genuinely reached and altered the real call every trial, with the banking backend's own destination-account validation independently catching it every time (a defense-in-depth result, not a plugin failure); message spoofing, prompt injection, and the malformed-failure control show a genuine, verified mutation reaching the real model and the model consistently declining to act on it, 5/5 for each. The extended scenario library (Section VII) widens this evidence to 123 scored live trials across 26 attack and fault variants without changing the architecture.

V. Defenses: Security Mechanism Plugins and Zero Trust

Six security-mechanism plugins now cover every category a defense-in-depth testbed should, each registered through the identical apply(ctx) -> HookResult contract an attack plugin uses and classified in the trace as a distinct POLICY_EVENT rather than misclassified as an attack. AmountLimitGuardrailPlugin denies a tool call whose declared amount exceeds a configured limit; live-verified blocking a tool-mutation attack that inflated a transfer from $125.50 to $5,000 before it reached the banking backend. RiskAwareRoutingGuardPlugin independently re-fetches a transaction's real risk score from the banking backend rather than trusting the router's own decision, closing route confusion – the one attack that succeeded 5/5 undefended – live-verified 5/5 deny and, with an optional redirect_to recovery mode, 5/5 reaching the identical manual_review ground truth the undefended baseline itself asserts. ResponseRedactionPlugin masks account-number-shaped tokens at the output control point; live verification surfaced and fixed a genuine Unicode-hyphen gap in its own detection regex.

BatchIntegrityGuardPlugin closes the back office's own undefended ledger-mutation attack by independently rechecking a target batch's real expected-vs-posted totals, live-verified 5/5 with a control run confirming no false positive on the legitimate flow. RateLimitGuardrailPlugin caps calls per run/window/argument value, turning a hijack-induced crash (0/10 undefended) into 5/5 completed runs. ActionIsolationPlugin refuses tools with a chosen side effect regardless of caller, denying an attacked funds transfer in 5/5 trials with the backend's transaction count unchanged.

Zero Trust Backplane. MANTIS integrates the real, independently-developed Citi_P3_Zero_Trust_Backplane project's Task 4 signed Policy Decision Point [4] – not a reimplementation – as a toggleable extension. A stock install never imports it; zero_trust_enforcement registers in the same plugin registry every other defense uses, guarded by a single try/except ImportError, so the optional dependency's absence means the plugin is genuinely unregistered rather than registered-but-broken. A dedicated zero_trust: {enabled: true, pdp_url: ...} block (or the equivalent checkbox in the minimal UI) turns it on or off per run; MANTIS's own contribution is extensions/zero_trust/generate_policy.py, which reads the testbed's live inventory and emits a default-deny-by-domain policy (an agent may act on a tool in its own domain; anything else is denied by the PDP's own default) in the real project's schema. Live verification confirmed same-domain allow, cross-domain deny, an unrecognized agent denied before a challenge is even issued, a shared tool name (search_policies, front and mid office) resolving correctly in both directions, and two full legitimate workflow runs through the real PDP completing identically to the undefended baseline. Eleven regression tests guard this integration against future drift, verified by deliberately injecting two kinds of drift and confirming both were caught before reverting. This paper's own AML office (Section VI) is registered in the same domain inventory this policy generator reads from, so the generated policy now also covers a fourth domain without any change to the generator itself.

VI. The AML Office: Porting Anti-Money Laundering from Citi_P3

A. Motivation and Scope

MANTIS's banking agents were extracted from a larger legacy monorepo, Citi_P3 [3], during the original migration documented in MIGRATION.md. That legacy codebase's citi_banking_adk package included a working AML investigation office – an investigator agent, a SAR drafting agent, and four real detection functions – that was never carried over when the rest of the codebase was translated into MANTIS's modular, hook-bus-instrumented architecture. This section reports that port: the detection logic is adapted, not reinvented, with the mathematics and control flow of every detection function unchanged from the source; MANTIS-side work is confined to the toggle mechanism, the agent-tree wiring, and the data-access layer the ported functions call into.

B. Detection Logic

Four detection functions are ported with their logic unchanged. Structuring detection (detect_structuring) evaluates a sliding window over a customer's inbound deposit-like transactions (by transaction type or by destination-in-customer-accounts/source-not-in-customer-accounts), accumulating a rolling total and reporting a hit whenever that total, within a configurable window (default 7 days), meets or exceeds a configurable threshold (default $10,000); a genuine hit creates a persisted AML alert and, optionally, a manual review case. Threshold-anomaly checking (check_threshold_anomaly) flags a transaction exceeding 5× a customer's baseline peak transfer amount. Mule-ring detection (find_mule_rings_for_customer) builds a directed graph from a customer's own transaction flows and reports closed loops via graph cycle detection, scoped – as in the source – to ring segments where the customer's own accounts appear as an endpoint. Identity-document verification (verify_identity_document) scores a submitted name (Unicode-normalized, case-folded, Levenshtein similarity), date of birth (exact calendar-date match), and ID number (exact string match) against reference values, weighted 0.30/0.30/0.40, returning a bounded, deterministic confidence score. Separately, an alert-lifecycle state machine (open → in-investigation → cleared/escalated-to-sar) is ported from the legacy codebase's standalone backend service into BankingRepository, the same SQLite-backed data-access layer every other banking tool already uses, with three new tool wrappers (get_aml_alert, list_aml_alerts, transition_aml_alert_status) giving an agent a real path to use it.

C. Toggle and Packaging

The AML office follows the same two-layer toggle discipline Section V established for Zero Trust, adapted to a domain rather than a policy plugin. Per-run: a dedicated aml_office: {enabled: true} configuration block (mirroring zero_trust:'s shape) controls whether the root agent's sub_agents list includes the AML office router; every configuration without that block produces a root agent whose sub_agents, instruction, and description are byte-identical to before this domain existed, verified directly by a regression test constructing the agent tree both ways and diffing it. Packaging: mule-ring detection's graph-cycle dependency (networkx) is an optional extra (pip install -e ".[aml]"), imported lazily inside the one function that needs it, so every other AML tool, and mantis –inventory/–validate, work in a stock install; calling the mule-ring tool without the dependency raises a clear,

actionable ImportError rather than failing silently or crashing tool discovery.

D. Live Verification: Two Real Defects Found and Fixed

Five scenarios exercise the AML office end to end against the live backend and a real model (gpt-oss-20b via UF Navigator): a structuring-detection positive case (three seeded inbound wires from one counterparty summing to $12,000 within seven days, past the $10,000 threshold), a structuring negative control, a mule-ring negative control, an identity-verification case, and an alert-triage case against a seeded reference alert. The first live pass of all five surfaced two real integration defects, neither caught by the unit test suite because both are properties of the live agent tree, not of the ported functions in isolation – the same class of finding both prior papers report as their central methodological lesson, that a clean exit or a passing unit test does not establish that a capability is reachable in the actual running system.

Routing gap. The identity-verification scenario was misrouted to front_office_router instead of aml_office_router: the root agent's routing instruction mentioned "AML investigation, suspicious activity, and SAR drafting" but not identity verification by name, and a plausible-sounding KYC-flavored prompt was reasonably routed to the front office instead, where it called get_customer_context with the submitted ID number in place of a customer id and produced a nonsensical result. The fix was wording, not logic: the root agent's and the AML router's instructions now explicitly name identity-document verification and alert triage as AML-office tasks.

Unreachable capability. The alert-triage scenario asked to transition a seeded alert's status; transition_aml_alert_status existed, was correctly implemented, and was unit-tested directly as a function – but no agent in the built tree had it, or its sibling read tools, on its toolset at all. The three alert-lifecycle tools were wired into the existing aml_investigator agent's toolset, the single tool-using agent in this domain, and its instruction extended to describe the triage capability.

Both fixes verified live after correction: the identity-verification scenario now routes correctly to aml_office_router → aml_investigation_workflow, calls verify_identity_document with the exact submitted/reference values from the prompt, and reports 100% match confidence. The alert-triage scenario now calls transition_aml_alert_status, and we confirmed the effect directly against the database rather than trusting the model's own report: reading the alert back afterward shows its status genuinely changed from open to in-investigation. All five scenarios were then run officially from committed configuration files (configs/extensions/aml_office_*_demo.yaml) and scored by mantis –evaluate: tool_use_correctness 1.0 and workflow_outcome 1.0 (completed) for every one of the five, with zero missing expected tools and zero forbidden tools used. Table I summarizes all five after both fixes.

TABLE I
AML office live verification, all five scenarios, against a real model after the fixes in § VI-D.

Scenario	Tool called	Observed
Structuring (positive)	detect_structuring	Correctly detected $12,000 cumulative over 7 days; real alert and manual-review case persisted.
Structuring (routine control)	detect_structuring	Correctly reported no structuring; no alert created.
Mule ring (control)	find_mule_rings_for_customer	Correctly reported no closed loop in the seeded history.
Identity verification	verify_identity_document	Correctly routed after fix; 100% match confidence on identical submitted/reference values.
Alert triage	transition_aml_alert_status	Correctly routed after fix; database re-read confirms the real status transition.

E. An Honest Limitation Inherited, Not Introduced

The ported detect_structuring function is deterministic and, as Table I shows, correct. The investigator agent's own judgment on top of a correct detection is an LLM call, and its gating logic for whether to draft a SAR (risk score ≥ 50, or an investigation status containing "suspicious") is inherited unchanged from the source prompt. A live run against the positive structuring case correctly identified the pattern in its findings text but assigned a risk score of 7.2 – well under the reporter's trigger threshold – and, separately, populated its case_facts with invented transaction amounts and dates rather than the real values the tool had just returned. We report this as a known limitation of that prompt against this specific smaller model, inherited directly from Citi_P3 and not something this port introduced or silently corrected, consistent with both prior papers' practice of reporting a defense's real scope rather than overstating it.

VII. Additional Workloads, UI, and Scenario Library

Three additional banking workloads, each backed by real persisted records rather than a new prompt into an existing process: dispute filing (front office), SAR/exception escalation (back office), and loan pre-approval (mid office), the last computing a real, deterministic decision from seeded account data rather than an LLM opinion. A second institution profile (regional_credit_union) reuses every agent, tool, and workflow unchanged against an isolated database and a genuinely stricter policy threshold (10% of balance versus 25%), selected by a runtime configuration field. A minimal, schema-driven browser UI (mantis –ui) provides an experiment editor and trace viewer whose every mutating action – validate, run, campaign – shells out to the identical mantis CLI subprocess a terminal user would invoke, verified by confirming a deliberately invalid configuration returns the real CLI's

TABLE II
Complete-system live-verification evidence.

Capability	Live evidence
Attacks/failures (5)	Route confusion 5/5 fired+effect; tool mutation genuinely reaches and alters the real call every trial; prompt injection, message spoofing, malformed-failure control genuinely reach the model/agent 5/5 with the agent correctly resisting each.
Defenses (6)	Every attack that once succeeded 5/5 undefended now has a live-verified 5/5 defense; no false positive on any paired no-attack control.
Zero Trust Backplane	Same-domain allow, cross-domain deny, unrecognized-agent pre-challenge deny, two full legitimate runs through the real PDP identical to baseline.
AML office (new)	All 5 scenarios correct after fixing 2 real integration defects found via live verification (Table I); one transition confirmed against the database, not just the model's report.
Additional workloads (3)	Live, 5/5 each: real persisted dispute, SAR, and loan-decision records.
Institution variant	Live, 5/5: identical code path, isolated data, genuinely different real decision.
Minimal UI	Real invalid config rejected with the real CLI's own error text; real server confirmed via HTTP.
Scenario library	123 scored live trials across 26 attack/fault variants; 5 more for the AML office.

own error text rather than a UI-side check; dedicated toggle fieldsets for Zero Trust and the AML office sit alongside the generic attack/policy dropdowns. An extended scenario library ships 19 measured no-attack baselines and 26 attack/fault variants (123 scored live trials across five families), and a from-scratch walkthrough adding a new attack plugin in about fifteen lines. Five starter campaigns – the original four [2] plus a fifth added for this paper's own AML office, sweeping its five scenarios with no attack configured and live-verified at tool_use_correctness 1.0 across all five – each a single mantis –campaign command producing a consolidated report. The AML office extends the scenario library by five further scenarios under its own domain (Section VI), each reproducible from a committed configuration file.

VIII. Evaluation Summary

The offline test suite – unit and contract tests, a parametrized consistency suite over every shipped configuration and scenario, and regression guards for every toggleable extension – stands at 853 passing tests with neither optional dependency (zero_trust, aml) installed, 880 with both installed, zero failures in every combination, and 178 more skipped by design without a live backend or model. This paper's own AML office extension adds 53 of those tests (repository-layer, detection-function, toggle, and drift-regression coverage) on top of the base established by [1] and the fourteen further categories added by [2]. Table II consolidates live-verification evidence across the complete system.

IX. Conclusion

This paper consolidates MANTIS as a complete system: the original five-control-point testbed [1], all five of its deferred extensions [2] including a real, toggleable Zero Trust Backplane integration, and – new here – a toggleable AML office porting real structuring, mule-ring, and identity-verification detection logic from the legacy Citi_P3 codebase MANTIS was originally extracted from. Every extension across all three papers shares the same discipline: a capability is reported as working only once verified against the system's actual emitted trace or persisted database state, not a clean exit code or a passing unit test in isolation. The AML office extension is itself a fresh instance of that discipline's value: live verification against five new scenarios found two real defects – a routing-instruction gap and an unreachable, unwired tool capability – that no unit test caught, because both are properties of the live agent tree rather than of any single ported function, and both were fixed and re-verified, including one effect confirmed directly against the database rather than accepted from the model's own report. The complete offline suite passes with zero failures across all four combinations of its two optional dependencies (853 with neither, 880 with both), and every live capability this paper reports – attacks, defenses, Zero Trust, the three additional workloads, the institution variant, the UI, and now the AML office – is reproducible from a committed configuration file against the real backend and a real model, consistent with the evidentiary standard both prior papers set for themselves.

Acknowledgment

References

[1] [Authors], "MANTIS – Modular Agent Network Testbed for Instrumentation and Security in Multi-Agent Banking Systems," 2026.
[2] [Authors], "MANTIS Extended – Post-Paper Extensions to a Banking Multi-Agent Security Testbed," 2026.
[3] smartsystemslab-uf, "Citi P3," github.com/smartsystemslab-uf/Citi_P3, 2026. (Legacy source repository MANTIS's banking agents were extracted from.)
[4] smartsystemslab-uf, "Citi P3 Zero Trust Backplane," github.com/smartsystemslab-uf/Citi_P3_Zero_Trust_Backplane, 2026.
[5] X. Zhang et al., "Enhancing Anomaly Detection in Financial Markets with an LLM-based Multi-Agent Framework," arXiv preprint arXiv:2403.19735, 2024.
[6] "MASCA: LLM based-Multi Agents System for Credit Assessment," arXiv preprint arXiv:2507.22758, 2025.
[7] D. Lee and M. Tiwari, "Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems," arXiv preprint arXiv:2410.07283, 2024.
[8] "Red-Teaming LLM Multi-Agent Systems via Communication Attacks," in Findings of the Association for Computational Linguistics: ACL 2025, arXiv preprint arXiv:2502.14847.
[9] "A Survey on Agentic Security: Applications, Threats and Defenses," arXiv preprint arXiv:2510.06445, 2025.
[10] "Security Considerations for Multi-agent Systems," arXiv preprint arXiv:2603.09002, 2026.
[11] "REDAgentBench: Executable Red Teaming and Faithful Measurement of LLM Agent Systems," arXiv preprint arXiv:2608.10669, 2026.

[12] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, Aug. 2020.
[13] "A Novel Zero-Trust Identity Framework for Agentic AI: Decentralized Authentication and Fine-Grained Access Control," arXiv preprint arXiv:2505.19301, 2025.